Projeto e Análise de Algoritmos

Prof. Hamilton José Brumatto

Bacharelado em Ciência da Computação - UESC

9 de novembro de 2010

Arvore Geradora Mínima

Contextualição

- Considere o seguinte problema: Uma universidade possui vários campi no estado da Bahia, e deseja interligá-los usando o menor custo de fibra.
- Este problema pode ser modelado através de um grafo não orientado, onde os vértices representam cada campus e as arestas as ligações entre os campi.
- Cada aresta deve ser ponderada pelo custo da ligação, ou seja, o custo da fibra ótica, neste exemplo.

Árvore Geradora Mínima

- Para resolver o problema, precisamos encontrar um SUBGRAFO GERADOR, *conexo* (para garantir a interligação entre todos os campi) e cuja soma dos custos de suas arestas seja o menor possível.
- O grafo original precisa ser conexo, senão não teríamos um subgrafo conexo. Vamos considerar que nosso problema sempre apresenta um grafo conexo.
- O subgrafo gerador procurado é uma árvore, pois se houvesse um ciclo significa que existem dois vértices com dois caminhos diferentes ligando-os, neste caso, eliminar um caminho tornaria um custo menor.

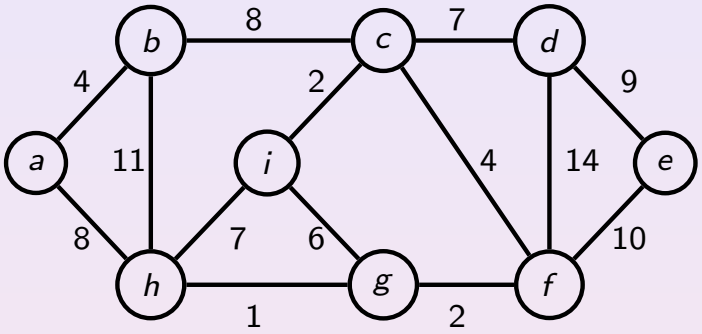
O Problema da Árvore Geradora Mínima

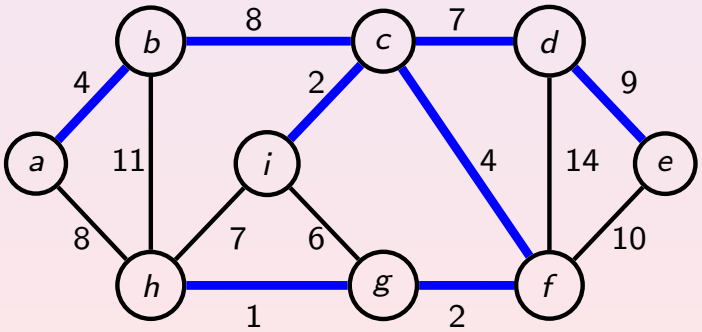
- Entrada: Grafo conexo $G = (V, E)$ com pesos $w(u, v)$ para cada aresta $\{u, v\}$.
- Saída: Subgrafo gerador conexo T de G, cujo peso total

$$w(T) = \sum_{(u,v) \in T} w(u, v)$$

seja o menor possível.

Exemplo


a
b
c
d
e
f
g
h
i
8
7
4
2
9
11
4
14
8
7
6
10
1
2


a
b
c
d
e
f
g
h
i
8
7
4
2
9
11
4
14
8
7
6
10
1
2

Estratégia gulosa

- De uma forma genérica, a abordagem no problema envolve um algoritmo guloso. A escolha da aresta que constrói a árvore é construída de forma incremental.
- O conjunto de arestas A escolhido mantém o seguinte invariante:
  - No início de cada iteração, A está contido em uma Árvore Geradora Mínima.
  - Em cada iteração, determina-se uma aresta (u, v) tal que $A' = A \cup \{(u, v)\}$ também satisfaz o invariante.
- A aresta determinada é chamada de *aresta segura*
- Dois algoritmos utilizam esta abordagem:
  1. Algoritmo de Prim
  2. Algoritmo de Kruskal

Algoritmo Guloso

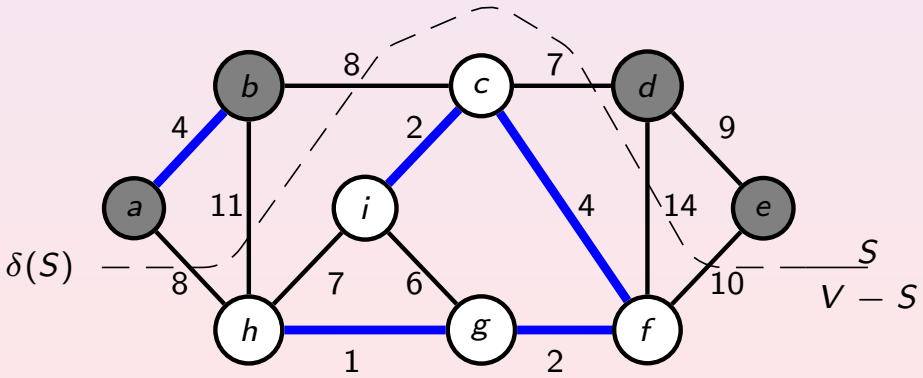
Algoritmo $AGM(G, w)$

1. $A \leftarrow 0$
2. **enquanto** A não é uma árvore geradora
3. **faça** Encontre aresta (u, v) segura para A
4. $A \leftarrow A \cup \{(u, v)\}$
5. **retorne** A

- O *algoritmo* está correto!
- Como o *laço* somente para se A for uma árvore geradora, então em cada iteração A está contido em uma AGM que chamaremos de T. Logo, existe uma *aresta segura* (u, v) em $T - A$.
- Para definir o algoritmo é necessário especificar como encontrar uma "aresta segura".

Encontrando uma aresta segura

- Vamos considerar o grafo $G(V, E)$ e tomamos $S \subset V$.
- Considere $\delta(S)$ o corte de S, ou seja, o conjunto de arestas que têm um extremo em S e outro em $V - S$.
- Seja $A \subset E$ contido em T, uma árvore geradora mínima.
- $\delta(S)$ respeita A se $A \cap \delta(S) = \emptyset$. Ou seja, o corte de S não contém nenhuma aresta de A.

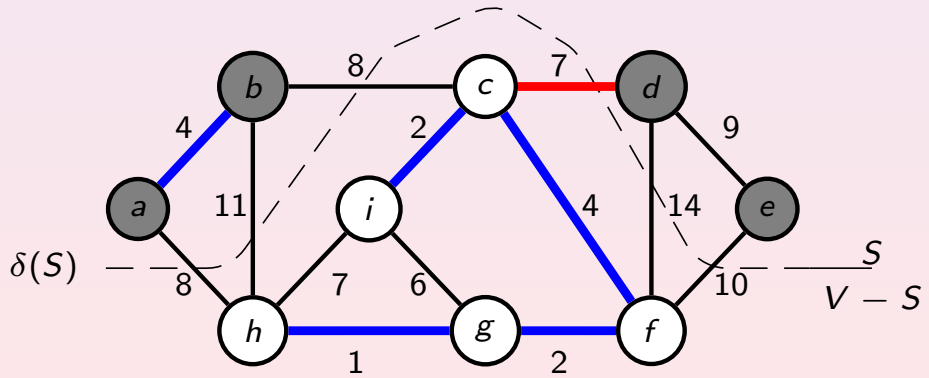


Achando uma aresta segura

Uma aresta do corte é *leve*, se tiver o menor peso entre as arestas.

Teorema

Seja G um grafo com pesos nas arestas dado por w. Seja A um subconjunto de arestas contido em uma Árvore Geradora Mínima. Seja $\delta(S)$ um corte que respeita A e (u, v) uma aresta leve deste corte. Então (u, v) é uma aresta segura.



Achando uma aresta segura

Demonstração.

Vamos supor T uma árvore geradora mínima e $A \subset E(T)$, com $(u, v) \notin E(T)$. vamos tomar P o único caminho de u a v em T. escolhemos

$$(x, y) = \{e : e \in E(P) \cap \delta(S) | w(x, y) = \max(w(e))\}$$

uma aresta do corte $\delta(S)$ em P. Vamos fazer $A' = E(T) - (x, y) \cup \{(u, v)\}$. Ficamos com: $w(A') = w(E(T)) - w(x, y) + w(u, v)$, como (u, v) é leve no corte, então $w(u, v) \leqslant w(x, y)$. Logo: $w(A') \leqslant w(E(T))$. $T' = G(V, A')$ é também uma árvore geradora mínima e contém a aresta (u, v). Como $(x, y) \notin A$, $A \cup \{(u, v)\} \subset E(T')$. (u, v) é aresta segura. □

Algoritmos Prim e Kruskal

- Ambos algoritmos utilizam esta estratégia gulosa. Eles se baseiam no seguinte corolário ao teorema anterior:

Corolário

Seja G um grafo com pesos nas arestas dado por w. Seja A um subconjunto de arestas contido em uma Árvore Geradora Mínima. Seja C um componente (árvore) de $G_A = (V, A)$. Se (u, v) é uma aresta leve de $\delta(V(C))$, então (u, v) é segura para A.

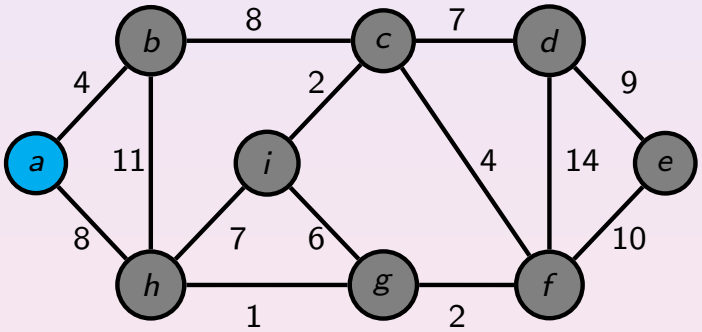
Demonstração.

Imediato a partir do teorema. □

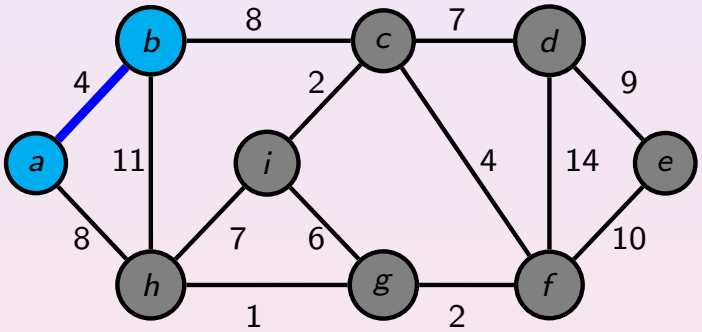
Aplicando a escolha gulosa no corte

- A é o conjunto de arestas de uma árvore com raiz r (escolhido arbitrariamente no início).
- Inicialmente $A = \emptyset$
- A cada iteração, o algoritmo define o corte $\delta(V(C))$, onde C é o conjunto de vértices definido pela árvore de raiz r.
- Ele encontra uma aresta leve (u, v) neste corte e acrescenta-a ao conjunto A, agregando o extremo de (u, v) que não pertença à árvore com raiz em r como nova folha da árvore.
- O algoritmo repete a iteração até que $T = T(C, A)$ seja uma árvore geradora do grafo original, neste caso, $V \equiv C$.
- A dificuldade no algoritmo é encontrar eficientemente uma aresta leve no corte.

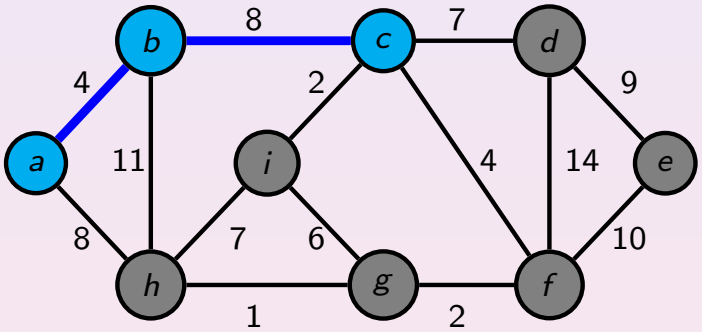
Aplicando o algoritmo



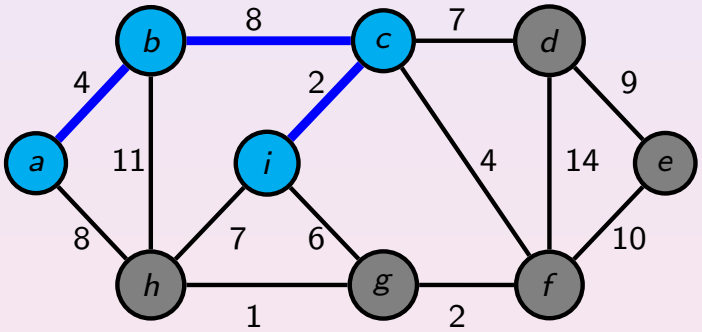
Aplicando o algoritmo



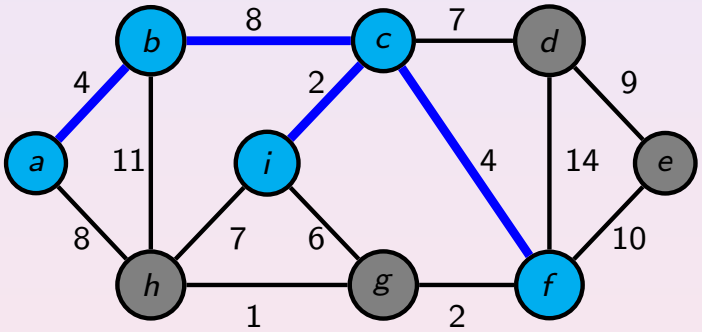
Aplicando o algoritmo



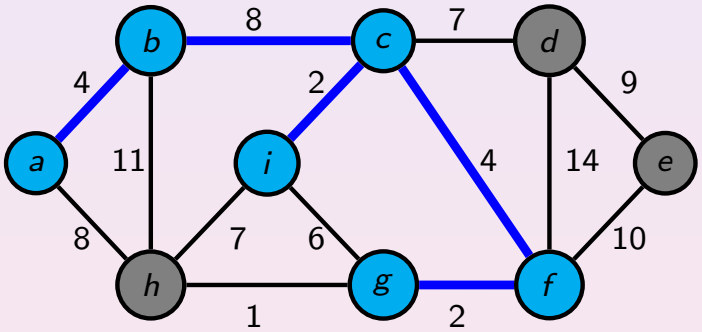
Aplicando o algoritmo



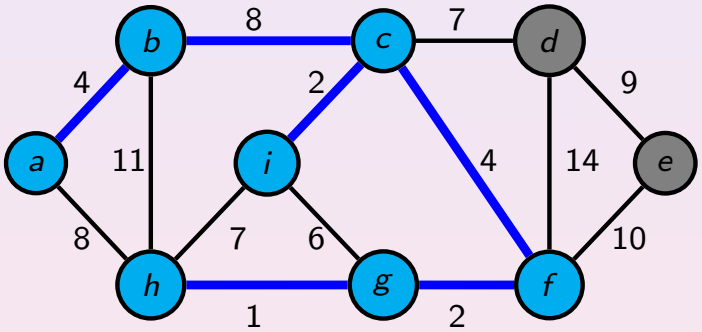
Aplicando o algoritmo



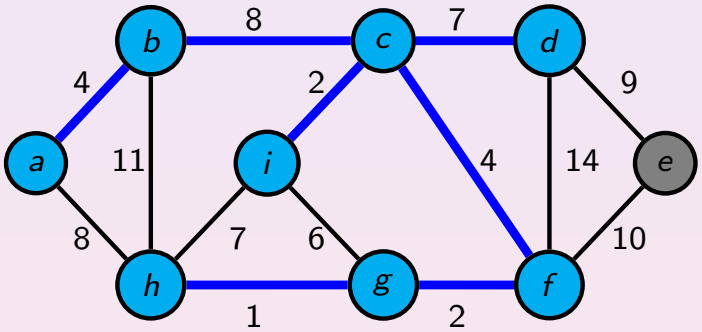
Aplicando o algoritmo



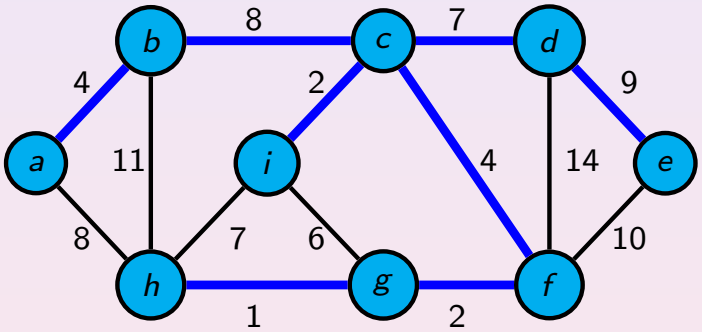
Aplicando o algoritmo



Aplicando o algoritmo



Aplicando o algoritmo



Características do Algoritmo

Informações definidas no algoritmo:

- Fila Q com todos os vértices
- Cada vértice v em Q tem uma chave $key[v]$ que indica o menor peso de qualquer aresta que o liga a algum vértice da árvore. Se não existir, $key[v] = \infty$
- A variável $\pi[u]$ indica o pai de u na árvore.
- A árvore $T = T(C, A)$ é definida pelo conjunto de arestas $A = \{(u, \pi[u]) : u \in V - \{r\} - Q\}$

O algoritmo

Algoritmo $Prim(G, w, r)$

1. **para** $u \in V(G)$
2. **faça** $key[u] \leftarrow \infty$
3. $\pi[u] \leftarrow nulo$
4. $key[r] \leftarrow 0$
5. $Q \leftarrow V(G)$
6. **enquanto** $Q \neq \emptyset$
7. **faça** $u \leftarrow ExtrairMinimo(Q)$
8. **para** $v \in Adj[u]$
9. **faça se** $v \in Q$ **e** $w(u, v) < key[v]$
10. **então** $\pi[v] \leftarrow u$
11. $key[v] \leftarrow w(u, v)$

Corretude do Algoritmo

Invariantes:

- No início de cada iteração do laço 6–11:
  - $A = \{(u, \pi[u]) : u \in V - \{r\} - Q\}$.
  - O conjunto de vérrtices da árvore é exatamente $V[G] - Q$.
  - Para cada $v \in Q$, se $\pi[v] \neq nulo$, então $key[v]$ é o peso de uma aresta $(v, \pi[v])$ de menor peso ligando v a um vértice $\pi[v]$ na árvore.
- Esse invariantes garantem que o algoritmo sempre escolhe uma aresta segura para acrescentar a A e portanto, o algoritmo está correto.

Complexidade do Algoritmo

- A complexidade do algoritmo depende de como a fila Q é implementada.
- Considere Q como um min-heap.
- As linhas 1–5 podem ser executadas em tempo $O(V)$.
- O laço da linha 6 é executado $|V|$ vezes e cada operação *ExtrairMinimo* consome tempo $O(\log V)$, resultando em um tempo total $O(V \log V)$ para todas as chamadas de *ExtrairMinimo*.
- O laço das linhas 8–11 é executado $O(E)$ vezes no total.
- Ao atualizar a chave de um vértice na linha 11 é feita uma atualização na fila que consome tempo $O(\log V)$.
- O tempo total é $O(V \log V + E \log V) = O(E \log V)$.

Atividades baseadas no CLRS

1. Ler capítulos 23, 23.1 e 23.2.
2. Exercícios: 23.1-1, 23.1-2, 23.1-3, 23.1-4, 23.1-5, 23.1-7, 23.1-8, 23.2-2, 23.2-8